

Desarrollo Web en Entorno Cliente

Examen — Convocatoria de Junio

PROCESO DE DESARROLLO

A partir de los archivos **index.html** y **styles.css** proporcionados, implementar la siguiente aplicación web en Angular:

1. Actividad 1

Tarea: Crear las vistas **View1**, **View2**, **View3** y el componente **header**, desde el cual se podrá navegar a las tres vistas creadas. Configura el enrutamiento en **app-routing.module.ts** y declara las rutas necesarias.

(0,5 puntos)

2. Actividad 2 — Pokémon (PokeAPI)

Tarea: Crea los componentes **PokemonCard** y **PokemonDetail** e instáncialos en la vista **View1**.

En la vista View1 implementa un formulario que permita al usuario introducir el nombre de un Pokémon. Cuando el usuario pulse sobre el botón **Enviar**, se borrará la información del formulario y se realizará una petición GET a la API REST pública **PokeAPI** (<https://pokeapi.co/api/v2/pokemon/{nombre}>) para obtener el nombre, el peso, la altura y los sprites del Pokémon indicado.

Para ello deberás implementar un **servicio** y las **interfaces** necesarias para gestionar la respuesta a la petición GET. Empleando **@Input**, **@Output** y **EventEmitter**, implementa la lógica necesaria para que el componente **PokemonCard** muestre: nombre, peso, altura y la imagen principal del sprite.

Al pulsar sobre la imagen del sprite en el componente **PokemonCard**, se activará el componente **PokemonDetail** en modo modal. Usando **@Input**, **@Output** y **EventEmitter**, este componente deberá mostrar las cuatro imágenes de sprites disponibles. El usuario podrá navegar entre ellas con los botones **anterior** y **siguiente** (pudiendo pasar de la primera a la última y viceversa). Si pulsa la **X**, el modal se cerrará. Si pulsa **volver**, regresará al formulario inicial.

Si el usuario realiza nuevas búsquedas, los resultados se acumularán, mostrándose los componentes **PokemonCard** de todos los Pokémon buscados.

(3 puntos)

3. Actividad 3 — Rick and Morty API

Tarea: Crea el componente **CharacterCard** e instáncialo en la vista **View2**.

Implementa la lógica necesaria para que al cargarse la vista se realice una petición GET a la API REST pública **The Rick & Morty API** (<https://rickandmortyapi.com/api/character>) para obtener los personajes de la primera página.

Para ello deberás implementar las funciones pertinentes en el **servicio** creado en la actividad anterior y las **interfaces** necesarias. Empleando **@Input**, **@Output** y **EventEmitter**, el componente **CharacterCard** mostrará inicialmente la imagen y el nombre del **primer personaje** de la página.

El usuario podrá navegar por los personajes de la página actual con los botones **anterior** y **siguiente** del propio componente (circular, primero↔último). Adicionalmente, en la vista View2 existirán dos botones **Página anterior** y **Página siguiente** para cambiar de página de la paginación de la API (también de forma circular, primera↔última).

La vista también ofrecerá un selector que permita filtrar entre mostrar personajes con estado **Alive**, **Dead** o **unknown**. Al cambiar el filtro se realizará una nueva petición a la API con el parámetro correspondiente y se reiniciará la navegación al primer personaje.

(3 puntos)

4. Actividad 4 — Star Wars API (SWAPI) + Free Dictionary API

Tarea: Crea los componentes **StarWarsCard** y **DictionaryResult** e instáncialos en la vista **View3**.

En View3 implementa un formulario con dos campos: uno para el **nombre de un personaje de Star Wars** y otro para una **palabra en inglés**. Habrá un botón **Enviar** por cada campo.

Al enviar el nombre del personaje, se realizará una petición GET a **SWAPI** (<https://swapi.dev/api/people/?search={nombre}>). Deberás implementar las interfaces necesarias y mostrar mediante el componente **StarWarsCard** (usando **@Input**): nombre, altura, peso y la lista de títulos de las películas en las que aparece el personaje. Si se pulsa **nueva búsqueda**, el formulario se reinicia. Las búsquedas se acumularán en pantalla.

Al enviar la palabra en inglés, se realizará una petición GET a la **Free Dictionary API** (<https://api.dictionaryapi.dev/api/v2/entries/en/{palabra}>). Se mostrarán tres botones: **Definiciones**, **Sinónimos** y **Antónimos**. Al pulsar cada botón se mostrará la información correspondiente mediante el componente **DictionaryResult** (usando **@Input**). Debajo de los resultados habrá botones **Volver** (al menú de botones) y **Nueva búsqueda** (reinicia el formulario).

(3,5 puntos)

EVALUACIÓN

Esta prueba supondrá el **50%** de la nota de la convocatoria de junio.

OBSERVACIONES

- Se entregará en el campus virtual un único archivo comprimido con la carpeta **src** del proyecto Angular implementado.

- Los archivos a entregar deberán ceñirse estrictamente a lo que se pide en cada enunciado. Las funcionalidades no solicitadas o el uso de librerías de terceros supondrán una penalización.
- **La copia o plagio de contenido de compañeros, la utilización de herramientas de IA o el uso del móvil o auriculares durante la realización del examen conllevará suspender automáticamente la convocatoria de junio.**